Hash

Aula 12 - Estrutura de dados

Professor Daniel Marques

UEPB
Universidade
Estadual da Paraíba

1

Introdução

Introdução

Quando tratamos de estrutura de dados estamos sempre interessados na eficiência de operações fundamentais de coleções, como busca, inserção e remoção

```
public class Hash {

    public static void main (String [] args){

        int [] meuArray = new int [100];
        meuArray[10] = "computacao@uepb"; //add
        meuArray[10] = null;    //delete
        //insercao, remocao, acesso, em tempo O(1)
    }
}
```

- Nesse sentido, o array, embora seja uma estrutura elementar, é um excelente escolha para diversos cenários, pois nos fornece acesso, inserção e remoção em tempo O(1)
- Isto é, se soubermos o índice em que um elemento está, o tempo de acesso a esse elemento é extremamente eficiente

Introdução

Vamos analisar o cenário em que desejamos armazenar objetos do tipo Aluno

```
public class Aluno {

    private int matricula;
    private String nome;

    public int getMatricula() {
        return this.matricula;
    }

    public String getNome() {
        return this.nome;
    }
```

Para fins de simplificação, o objeto do tipo aluno possui dois atributos: matrícula e nome

Introdução

Vamos também assumir que a matrícula identifica unicamente um aluno do curso e que é um inteiro no domínio [0...1999]

Nesse material chamamos de chaves os atributos dessa natureza

```
public static void main(String[] args) {
    Aluno[] alunos = new Aluno[2000];

    Aluno joao = new Aluno(21, "João Arthur");

    // inserindo em tempo constante
    alunos[joao.getMatricula()] = joao;

    // buscando em tempo constante
    if (alunos[joao.getMatricula()] != null)
        ...

    // removendo em tempo constante
    alunos[joao.getMatricula()] = null
}
```

Dentro desse cenário, um array é uma estrutura muito adequada, porque podemos armazenar o aluno na posição do array cujo valor é o mesmo de sua matrícula

2

Tabela de Acesso Direto

Tabela de Acesso Direto

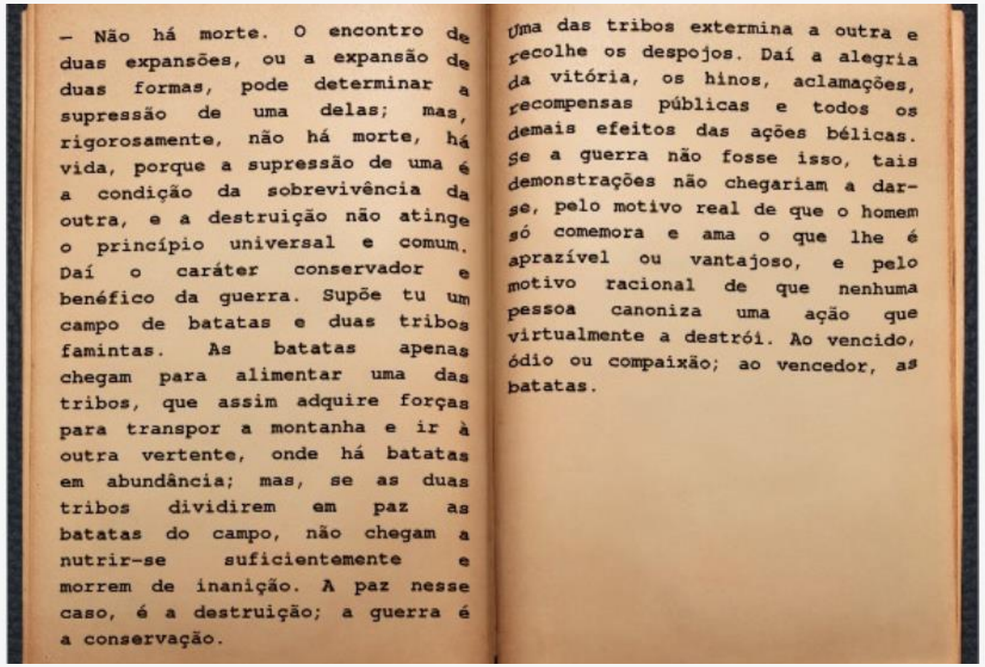
Note que, como há uma correspondência direta entre a matrícula e o índice em que o elemento está, as operações são muito eficientes.

```
public static void main(String[] args) {
    Aluno[] alunos = new Aluno[2000];

    Aluno joao = new Aluno(21, "João Arthur");

    // inserindo em tempo constante
    alunos[joao.getMatricula()] = joao;

    // buscando em tempo constante
    if (alunos[joao.getMatricula()] != null)
        ...

    // removendo em tempo constante
    alunos[joao.getMatricula()] = null
}
```

Chamamos de **Tabela de Acesso Direto** o array que é usado dessa maneira, isto é, com os valores sendo mapeados de forma direta para os índices do array

Tabela de Acesso Direto

Queremos contar o número de ocorrências de cada palavra de cada livro da biblioteca



— Não há morte. O encontro de duas expansões, ou a expansão de duas formas, pode determinar a supressão de uma delas; mas, rigorosamente, não há morte, há vida, porque a supressão de uma é a condição da sobrevivência da outra, e a destruição não atinge o princípio universal e comum. Daí o caráter conservador e benéfico da guerra. Supõe tu um campo de batatas e duas tribos famintas. As batatas apenas chegam para alimentar uma das tribos, que assim adquire forças para transpor a montanha e ir à outra vertente, onde há batatas em abundância; mas, se as duas tribos dividirem em paz as batatas do campo, não chegam a nutrir-se suficientemente e morrem de inanição. A paz nesse caso, é a destruição; a guerra é a conservação.

Uma das tribos extermina a outra e recolhe os despojos. Daí a alegria da vitória, os hinos, aclamações, recompensas públicas e todos os demais efeitos das ações bélicas. Se a guerra não fosse isso, tais demonstrações não chegariam a dar-se, pelo motivo real de que o homem só comemora e ama o que lhe é aprazível ou vantajoso, e pelo motivo racional de que nenhuma pessoa canoniza uma ação que virtualmente a destrói. Ao vencido, ódio ou compaixão; ao vencedor, as batatas.

- No idioma, há cerca de milhares de palavras (aproximadamente 435.000)
- Mas no total, há milhões de ocorrências!

Tabela de Acesso Direto

Queremos acessar uma palavra como se fosse um vetor:

```
ocorrencias["ilha"] = 8
```

dia:	6 ocorrências
escola:	13 ocorrências
gratuito:	1 ocorrência
ilha:	8 ocorrências
jeito:	5 ocorrências
lata:	2 ocorrências

- Vetor - acesso/escrita/inserção em O(n)
- Vetor ordenado - acesso/escrita em O(lg n)
  - inserir um novo leva O(n)
- Conseguimos fazer em O(1)?

Tabela de Acesso Direto

Se tivéssemos apenas uma palavra começando com cada letra era fácil

a	0	
b	1	
c	2	
d	3	dia
e	4	escola
f	5	
g	6	gratuito
h	7	
i	8	ilha
⋮	⋮	⋮

bastaria ter um vetor de 24 posições

Tabela de Acesso Direto

Mas, temos um problema. O que acontece quando temos palavras que começam com a mesma letra?

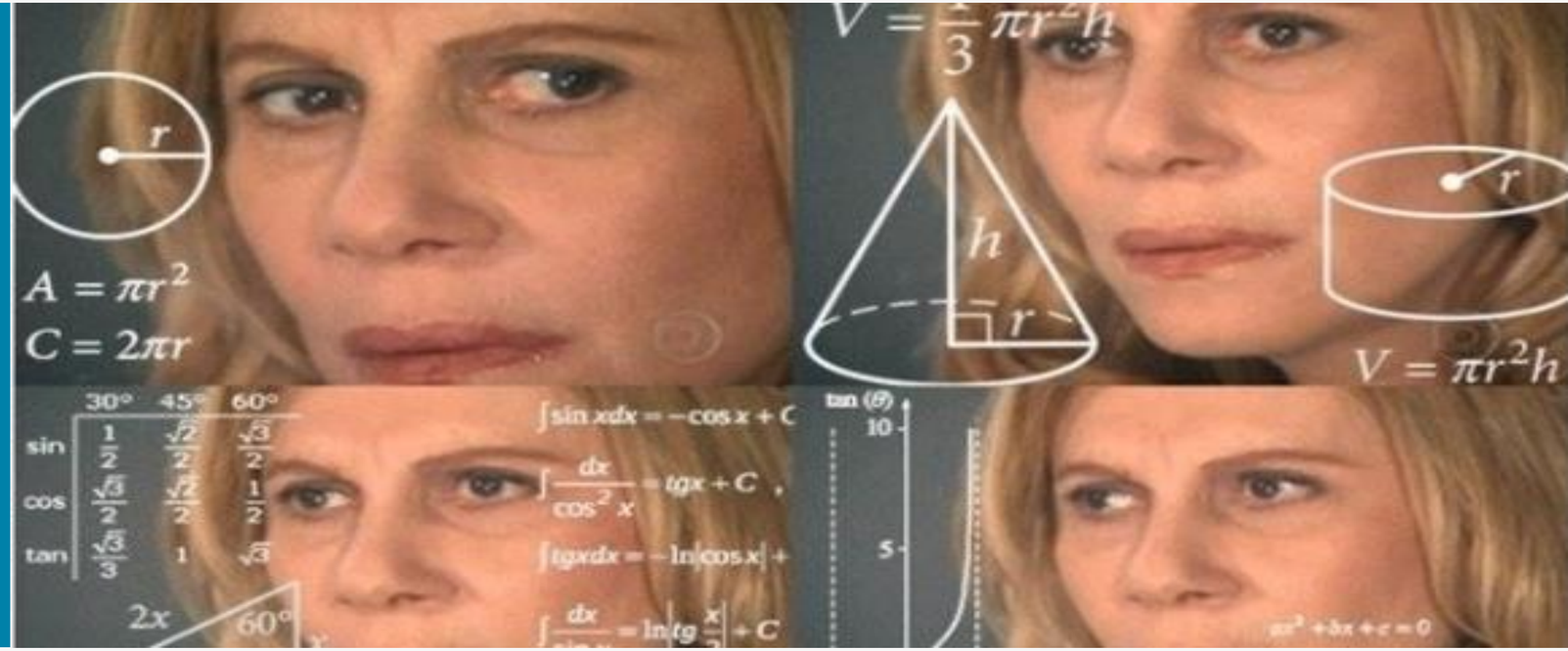


Tabela com lista encadeada

Ideia

Utilizar uma lista encadeada para cada letra

guardamos os ponteiros para as listas em um vetor

Tabela com lista encadeada

Inserindo “bala”

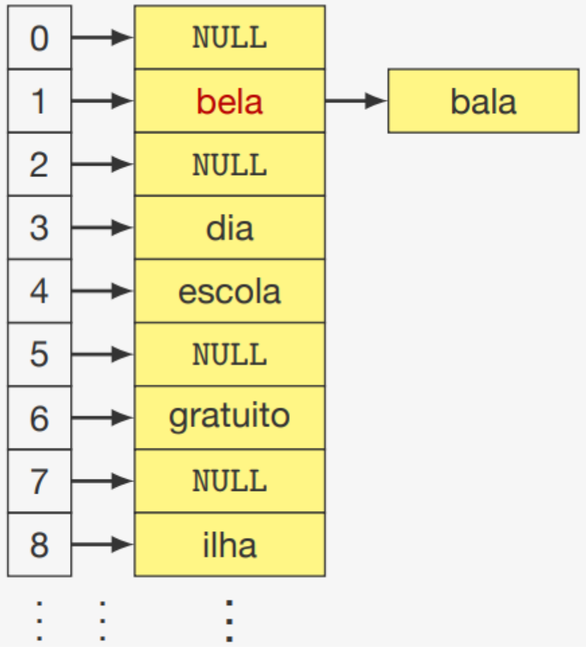
descobrimos a posição pela primeira letra

atualizamos o vetor para apontar para o nó de “bala”

Tabela com lista encadeada

Inserindo “bela”

Descobrimos a posição pela primeira letra

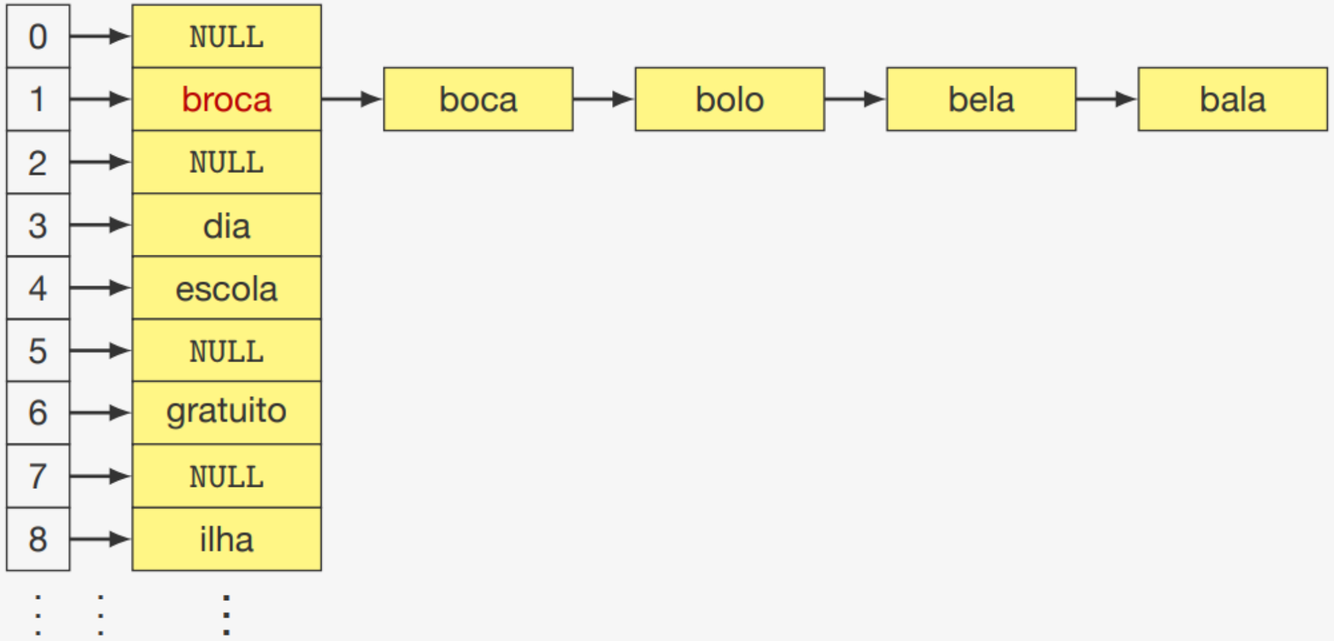


- temos uma **colisão** com “bala”
- inserimos no começo da lista da letra b

Tabela com lista encadeada

Após a inserção de várias palavras começando com b

- inserimos “bolo”, “boca”, “broca”



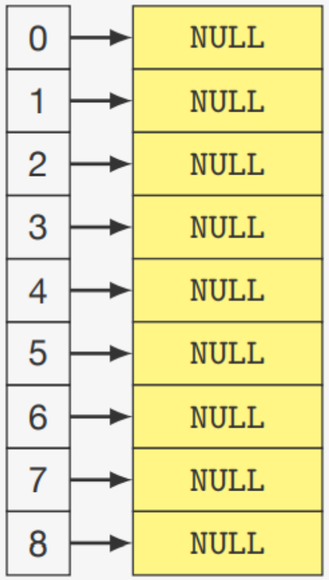
a tabela ficou degenerada em lista

Espalhamento com encadeamento separado

Vamos tentar espalhar melhor

Usamos um hash da chave (palavra)

broca
boca
bolo
bela
bala
dia
escola
gratuito
ilha



vamos associar a chave a um número inteiro (entre 0 e 8)

Espalhamento com encadeamento separado

Vamos tentar espalhar melhor

Usamos um hash da chave (palavra)

broca ⇝ `h("broca") = 3`

boca

bolo

bela

bala

dia

escola

gratuito

ilha

0	NULL
1	NULL
2	NULL
3	broca
4	NULL
5	NULL
6	NULL
7	NULL
8	NULL

vamos associar a chave a um número inteiro (entre 0 e 8)

Espalhamento com encadeamento separado

Vamos tentar espalhar melhor

Usamos um hash da chave (palavra)

broca ⇝ `h("broca") = 3`

boca ⇝ `h("boca") = 0`

bolo

bela

bala

dia

escola

gratuito

ilha

0	boca
1	NULL
2	NULL
3	broca
4	NULL
5	NULL
6	NULL
7	NULL
8	NULL

vamos associar a chave a um número inteiro (entre 0 e 8)

Espalhamento com encadeamento separado

Vamos tentar espalhar melhor

Usamos um hash da chave (palavra)

broca ⇝ h("broca") = 3

boca ⇝ h("boca") = 0

bolo ⇝ h("bolo") = 5

bela ⇝ h("bela") = 2

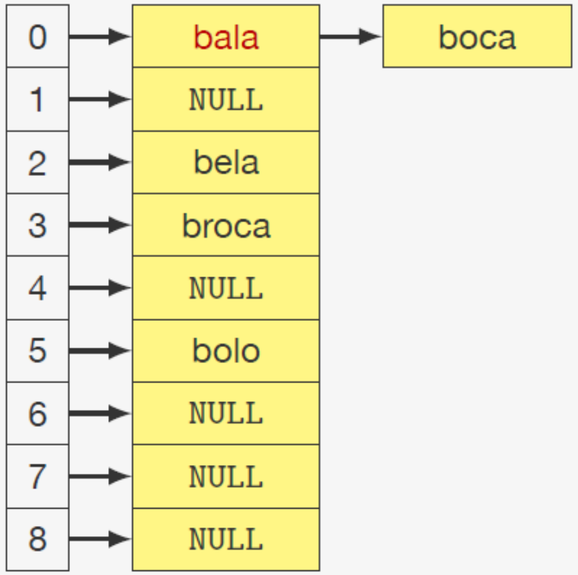
bala ⇝ h("bala") = 0

dia

escola

gratuito

ilha



vamos associar a chave a um número inteiro (entre 0 e 8)

Espalhamento com encadeamento separado

Vamos tentar espalhar melhor

Usamos um hash da chave (palavra)

broca ⇝ h("broca") = 3

boca ⇝ h("boca") = 0

bolo ⇝ h("bolo") = 5

bela ⇝ h("bela") = 2

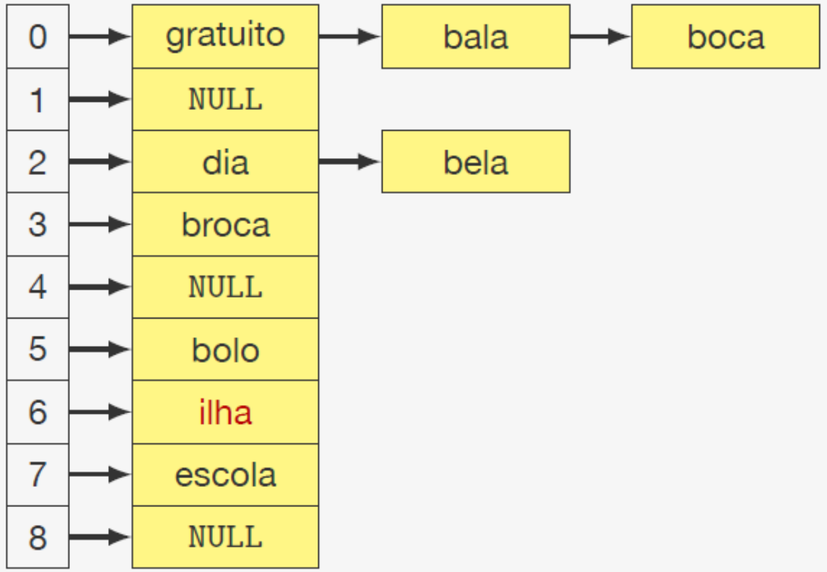
bala ⇝ h("bala") = 0

dia ⇝ h("dia") = 2

escola ⇝ h("escola") = 7

gratuito ⇝ h("gratuito") = 0

ilha ⇝ h("ilha") = 6



vamos associar a chave a um número inteiro (entre 0 e 8)

Tabela de espelhamento

Tabela Hash

Uma função de hashing associa um elemento de um certo conjunto (strings, números, arquivos, etc.) a um número inteiro de tamanho conhecido

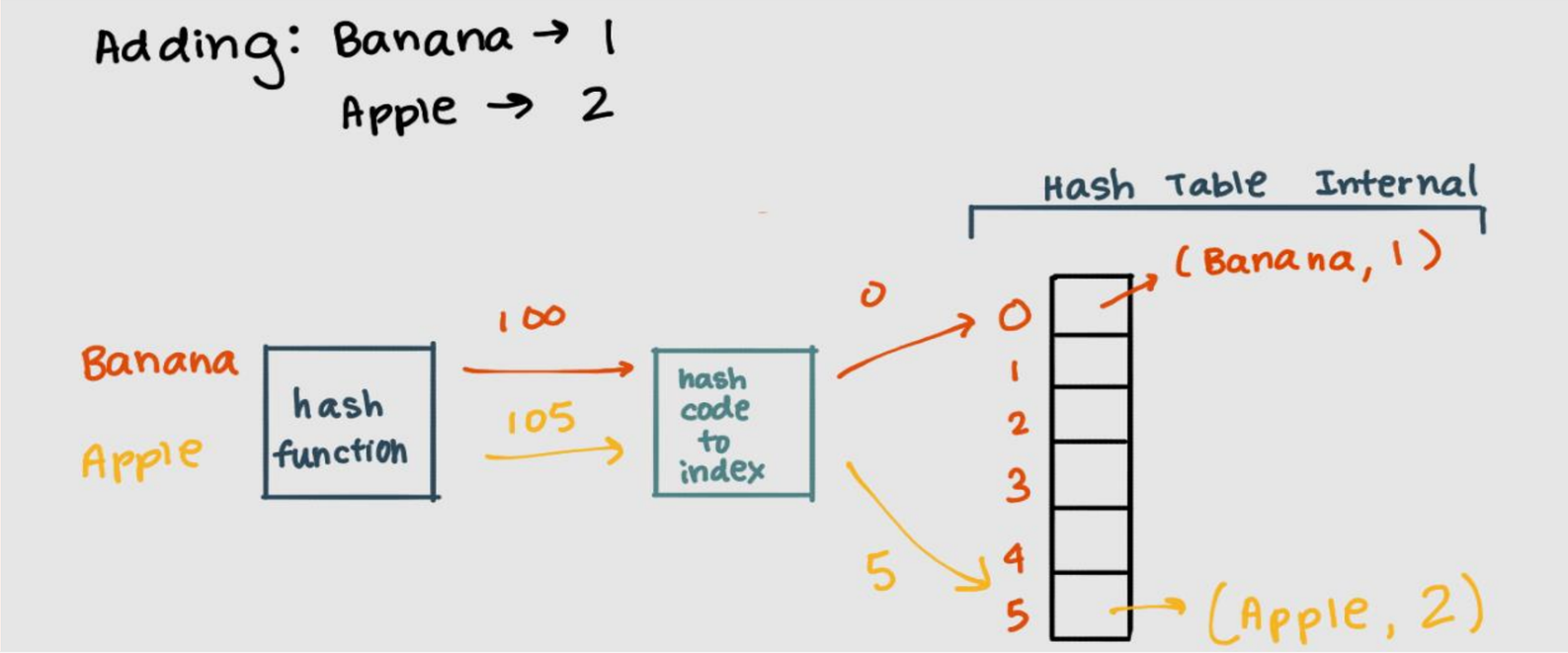
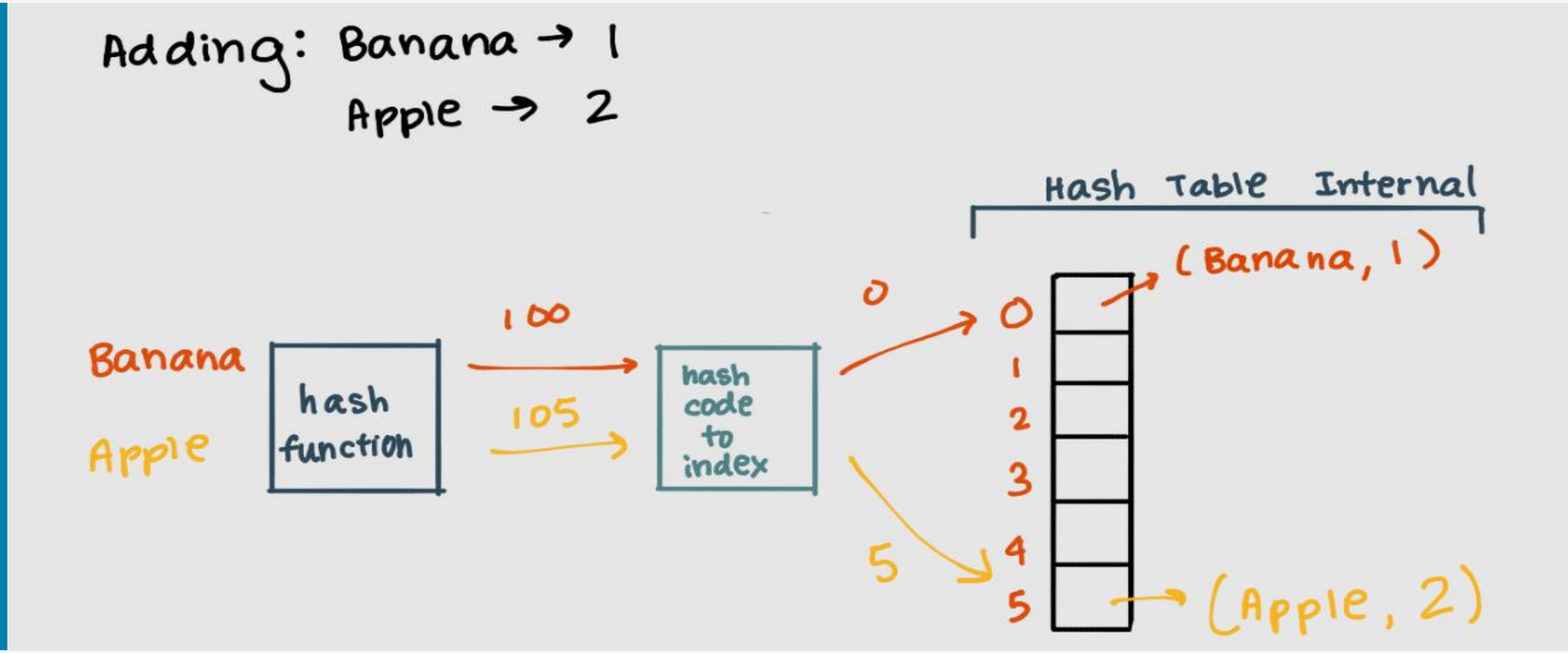


Tabela de espelhamento

Tabela Hash

Uma tabela de espalhamento é um tipo abstrato de dados para busca em conjuntos dinâmicos cuja implementação tem certas propriedades:



- Os dados são acessado por meio de um vetor de tamanho conhecido
- A posição do vetor é calculada por uma função de hashing

Características das tabelas de espelhamento

Tabela Hash

estimativa do tamanho do conjunto de dados deve ser conhecida

Tempo das operações depende da função de hashing escolhida:

- Chaves bem espalhadas: tempo “quase” O(1)
  - Se temos n itens e uma tabela de tamanho M
  - tempo de acesso é o tempo de calcular a função de hashing + O(n/M)
- Chaves agrupadas: pior caso de tempo O(n)
  - Vira uma lista ligada com todos os elementos

3

Função de Hash

Obtendo funções de hashing

Uma boa função de hashing espalha bem:

- A probabilidade de uma chave ter um hash específico é (aproximadamente) 1/M
- Ou seja, esperamos que cada lista tenha n/M elementos

Métodos genéricos (funcionam bem na prática):

- Método da divisão
- Método da multiplicação

Hashing perfeito

Se conhecermos todos as chaves a priori, é possível encontrar uma função de hashing injetora

Index	
0	
1	
-	
-	
-	
11	defabc
12	
13	
14	cdefab
-	
-	
-	
-	
23	bcdefa
-	
-	
-	
38	abcdef
-	

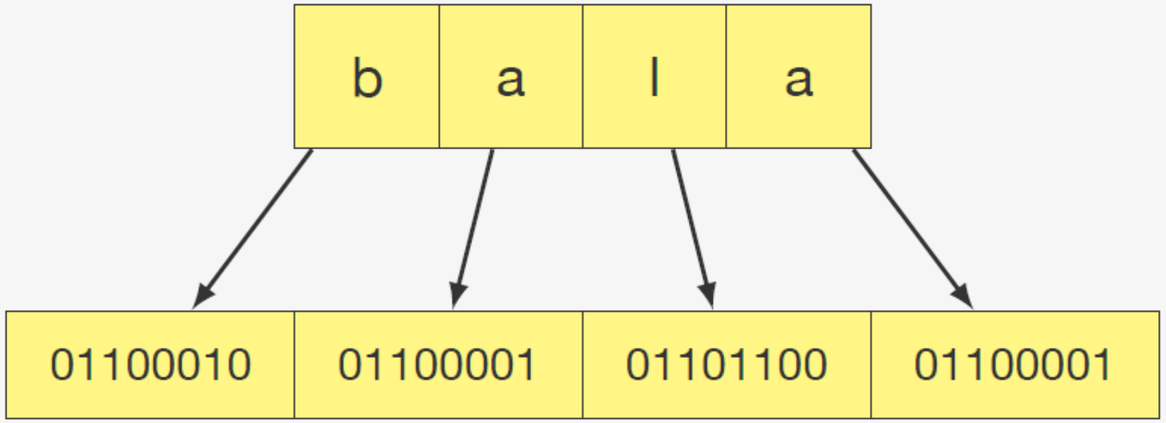
- Isto é, não temos colisões
- Tais funções podem ser difíceis de encontrar

Interpretando chaves

Pressupomos que as chaves são números inteiros

E se não for?

Reinterpretamos a chave como uma sequência de bits



Assim, “bala” se torna o número 1.650.551.905

Método da divisão

obtemos o resto da divisão pelo tamanho M do hashing

Exemplo:

$$h(\text{“bala”}) = 1.650.551.905 \textbf{ mod } 1783 = 277$$

- Escolher M como uma potência de 2 não é uma boa ideia, pois considera apenas os bits menos significativos
- normalmente escolhemos M como um número primo longe de uma potência de 2

Método da multiplicação

multiplicamos o valor de M por um certo valor real A e obtemos a parte fracionária

$$h(x) = \lfloor M\,(A \cdot x \textbf{ mod } 1) \rfloor$$

- Escolhemos A conveniente, por exemplo $A = (\sqrt{5} - 1)/2$
- posição relativa no vetor não depende de M (pode ser M = 1024)

Método da multiplicação

Exemplo

$$
\begin{aligned}
h(\text{“bala”}) &= \lfloor 1024 \cdot [((\sqrt{5}-1)/2 \cdot 1.650.551.905) \textbf{ mod } 1] \rfloor \\
&= \lfloor 1024 \cdot [1020097177{,}4858876 \textbf{ mod } 1] \rfloor \\
&= \lfloor 1024 \cdot 0{,}4858876 \rfloor \\
&= \lfloor 497{,}5489024 \rfloor = 497
\end{aligned}
$$

O uso da razão áurea como valor de A é sugestão de Knuth

Método do endereçamento aberto

Os dados são guardados no próprio vetor

Colisões são colocadas em posições livres da tabela

broca
boca
bolo
bela
bala
dia
escola
gratuito
ilha

0	
1	
2	
3	
4	
5	
6	
7	
8	

- Se a tabela encher, deve recriar uma tabela maior e mudar a função de hashing
- Remoção é mais complicada

Método do endereçamento aberto

Inserindo

broca ⇝ `h("broca") = 3`

boca

bolo

bela

bala

dia

escola

gratuito

ilha

0	
1	
2	
3	broca
4	
5	
6	
7	
8	

- Procuramos uma posição
- Se houver espaço, guardamos
- se não houver espaço, procuramos a próxima posição livre (módulo M)

Método do endereçamento aberto

Inserindo

Perceba que "bala" não pode ser inserido na posição 0, pois a mesma já está ocupada. Procure a primeira posição desocupada após a posição 0

broca ⇝ `h("broca") = 3`

boca ⇝ `h("boca") = 0`

bolo ⇝ `h("bolo") = 5`

bela ⇝ `h("bela") = 2`

bala ⇝ `h("bala") = 0`

dia

escola

gratuito

ilha

0	boca
1	
2	bela
3	broca
4	
5	bolo
6	
7	
8	

- Procuramos uma posição
- Se houver espaço, guardamos
- se não houver espaço, procuramos a próxima posição livre (módulo M)

Método do endereçamento aberto

Inserindo

O mesmo ocorre para os próximos casos de posição ocupada

broca ⇝ `h("broca") = 3`

boca ⇝ `h("boca") = 0`

bolo ⇝ `h("bolo") = 5`

bela ⇝ `h("bela") = 2`

bala ⇝ `h("bala") = 0`

dia ⇝ `h("dia") = 2`

escola ⇝ `h("escola") = 7`

gratuito ⇝ `h("gratuito") = 0`

ilha ⇝ `h("ilha") = 6`

0	boca
1	bala
2	bela
3	broca
4	dia
5	bolo
6	gratuito
7	escola
8	ilha

- Procuramos uma posição
- Se houver espaço, guardamos
- se não houver espaço, procuramos a próxima posição livre (módulo M)

Dúvidas???

Referências

1. BRUNET, João Arthur**. Tabelas Hash**. Universidade Federal de Campina Grande. 2019.

2. SCHOUERY, Rafael C. S.; **Tabela de Espelhamento**. Universidade Estadual da Campinas. 2017.

3.

4.

5.

UEPB
Universidade
Estadual da Paraíba